

**Trialbank**
Independent Trial Paper B

**2026**

**Higher School Certificate**

**Year 12 Trial HSC Examination**

# Software Engineering

**Total Marks: 80**

### General Instructions

- Reading time – 10 minutes
- Working time – 2 hours 20 minutes
- The HSC exam is completed online; answer on screen in Trialbank, or in the spaces provided on this paper
- Python is used for code questions
- Write your name at the top of every page

**Section I**
Total marks (20)

- Attempt Questions 1–16
- Allow about 25 minutes for this section

**Section II**
Total marks (60)

- Attempt ALL questions
- Allow about 1 hour and 55 minutes for this section

STUDENT NAME: _____

# Section I

**20 marks**

Attempt Questions 1–16

Allow about 25 minutes for this section

Questions 1–8: select ONE answer. Questions 9–12: select ALL correct answers (all must be correct for the mark). Questions 13–16: match each item to ONE option; options may be used more than once.

**1** **Which of these is a *non-functional* requirement for a school room-booking app?**

- ◯ **A.** Students can cancel a booking
- ◯ **B.** Every page loads in under 2 seconds
- ◯ **C.** Admins can export bookings to a spreadsheet
- ◯ **D.** Teachers can see who booked a room

**2** **A website stores passwords as SHA-256 hashes without a salt. Which attack does this make much easier?**

- ◯ **A.** A rainbow table attack
- ◯ **B.** Cross-site request forgery
- ◯ **C.** A denial-of-service attack
- ◯ **D.** Clickjacking

**3** **Which HTTP response header tells a browser to connect to the site only over HTTPS in future?**

- ◯ **A.** Content-Type
- ◯ **B.** Strict-Transport-Security
- ◯ **C.** Cache-Control
- ◯ **D.** Set-Cookie

**4** **What is the output of this code?**

```
def f(n):
    if n <= 1:
        return n
    return f(n - 1) + f(n - 2)

print(f(6))
```

- ◯ **A.** 5
- ◯ **B.** 8
- ◯ **C.** 13
- ◯ **D.** 6

**5** **Why is a dataset split into training data and testing data?**

- ◯ **A.** To make training faster
- ◯ **B.** To estimate how well the model performs on data it has not seen
- ◯ **C.** To reduce the number of features
- ◯ **D.** To label the data automatically

**6** **Which CSS selector targets elements with the class `card` that are inside a `<section>`?**

- ◯ **A.** `section.card`
- ◯ **B.** `section .card`
- ◯ **C.** `.card section`
- ◯ **D.** `section > card`

**7** **A login form says "Incorrect password" when the username exists but "User not found" when it doesn't. What vulnerability is this?**

- ◯ **A.** User enumeration (information disclosure)
- ◯ **B.** Cross-site scripting
- ◯ **C.** Buffer overflow
- ◯ **D.** Race condition

**8** **Which is the best example of MLOps practice?**

- ◯ **A.** Deploying a new model by hand once a year
- ◯ **B.** Automatically retraining, testing and deploying models, then monitoring them in use
- ◯ **C.** Writing all documentation after the project ends
- ◯ **D.** Testing a model only after customers report problems

**9** **Which of these are features of a RESTful API? Select ALL that apply.**

- ☐ **A.** Each request is stateless
- ☐ **B.** Resources are identified by URLs
- ☐ **C.** It requires WebSockets
- ☐ **D.** It uses HTTP methods such as GET and POST
- ☐ **E.** Responses are always HTML pages
- ☐ **F.** Responses are commonly JSON

**10** **Select ALL statements that are true for this code.**

```
class Vehicle:
    wheels = 4

    def __init__(self, make):
        self.make = make

    def describe(self):
        return f"{self.make} with {self.wheels} wheels"

class Motorbike(Vehicle):
    wheels = 2

bike = Motorbike("Yamaha")
car = Vehicle("Toyota")
```

- ☐ **A.** bike.describe() returns "Yamaha with 2 wheels"
- ☐ **B.** car.describe() returns "Toyota with 4 wheels"
- ☐ **C.** Motorbike must define its own __init__ before objects can be created
- ☐ **D.** wheels is a class attribute
- ☐ **E.** Motorbike overrides the describe() method
- ☐ **F.** Motorbike inherits __init__ from Vehicle

**11** **Which of these are ethical concerns with facial recognition systems? Select ALL that apply.**

- ☐ **A.** Lower accuracy for some demographic groups
- ☐ **B.** Collecting biometric data without consent
- ☐ **C.** Faster image loading
- ☐ **D.** Enabling mass surveillance
- ☐ **E.** Lower electricity use

**12** **Which are appropriate ways to store users' passwords? Select ALL that apply.**

- ☐ **A.** In plain text
- ☐ **B.** Hashed with bcrypt
- ☐ **C.** Hashed with Argon2
- ☐ **D.** Encoded in base64
- ☐ **E.** Hashed with MD5
- ☐ **F.** Hashed with PBKDF2 using many iterations

**13** **Match each type of testing to its description.** **2 marks**

| Item | Answer | Options |
|---|---|---|
| 1. Unit testing | | A. The whole system is tested against the requirements |
| 2. Integration testing | | B. The client confirms the system meets their needs |
| 3. System testing | | C. A single function or module is tested on its own |
| 4. Acceptance testing | | D. Modules are tested to check they work together |

**14** **Match each security principle to its description.** **2 marks**

| Item | Answer | Options |
|---|---|---|
| 1. Principle of least privilege | | A. Several layers of security, so one failure isn't fatal |
| 2. Defence in depth | | B. Checking data is the right type, length and format before using it |
| 3. Input validation | | C. Users and processes get only the access they need |
| 4. Session management | | D. Stored data is unreadable if a disk or database is stolen |
| 5. Encryption at rest | | E. Creating, tracking and expiring logged-in sessions securely |

**15** **Match each technology to its main role in a web app.** **2 marks**

| Item | Answer | Options |
|---|---|---|
| 1. HTML | | A. Querying and updating the database |
| 2. CSS | | B. The structure and content of a page |
| 3. JavaScript (in the browser) | | C. Server-side routes and logic |
| 4. SQL | | D. Presentation and layout |
| 5. Python with Flask | | E. Interactivity in the browser |

**16** **Match each machine learning term to its meaning.** **2 marks**

| Item | Answer | Options |
|---|---|---|
| 1. Feature | | A. One complete pass through the training data |
| 2. Label | | B. An input variable used to make a prediction |
| 3. Epoch | | C. The known output the model learns to predict |
| 4. Learning rate | | D. How big each weight update is |
| 5. Overfitting | | E. A non-linear function applied to a neuron's output |
| 6. Activation function | | F. Scoring well on training data but poorly on new data |

## Section II

**60 marks**

Attempt ALL questions

Allow about 1 hour and 55 minutes for this section

Answer each question in the spaces provided. Write code in Python unless a question says pseudocode is acceptable.

**17** Outline the purpose of adding a *salt* before hashing a password. **2 marks**

**18** Explain how a cross-site request forgery (CSRF) attack works, and describe ONE way to prevent it. **3 marks**

**19** This Flask route shows search results. **4 marks**

```
@app.route("/search")
def search():
    q = request.args.get("q", "")
    return f"<h1>Results for {q}</h1>"
```

(a) Identify the vulnerability and explain how an attacker could exploit it. **(2 marks)**

(b) Rewrite the route so it is secure. **(2 marks)**

**20** A student opens the profile page of a web app. Describe how the browser, the web server and the database work together to display the page. **4 marks**

**21** Compare progressive web apps (PWAs) with native mobile apps. **4 marks**

**22** `results` is a list of `(name, mark)` tuples. Write a Python function `top_students(results, n)` that returns a list of the names of the `n` highest-scoring students, highest mark first. Students with equal marks are listed alphabetically. If there are fewer than `n` students, return all of them. **5 marks**

For example, `top_students([("Ava", 80), ("Ben", 95), ("Cy", 80)], 2)` returns `["Ben", "Ava"]`.

**23** A database has these tables: **3 marks**

```
Students(id, name, year)
Bookings(id, student_id, room, day)
```

Write an SQL query that shows each room and how many times Year 11 students booked it, including only rooms booked more than 5 times, most-booked first.

**24** Explain why developers use branches in version control. **2 marks**

**25** A school uses a decision tree to predict whether a student will pass, based on attendance and homework completion. **4 marks**

(a) Explain how a decision tree makes a prediction for a new student. **(2 marks)**

(b) Give ONE advantage and ONE disadvantage of using a decision tree instead of a neural network for this task. **(2 marks)**

**26** Explain how bias can enter a machine learning system at different stages of its development and use. **4 marks**

**27** A spam filter is tested on 100 emails: 40 spam correctly flagged, 10 normal emails wrongly flagged as spam, 5 spam emails missed and 45 normal emails correctly allowed. Calculate the filter's accuracy and its precision. **2 marks**

**28** Write Python code for these two classes. **5 marks**

- **BankAccount** has a *private* balance starting at 0, and methods `deposit(amount)`, `withdraw(amount)` and `get_balance()`. Both `deposit` and `withdraw` raise a `ValueError` if the amount is not positive, and `withdraw` also raises one if the amount is more than the balance.
- **SavingsAccount** is a BankAccount created with an `interest_rate` (for example 0.05). Its method `add_interest()` deposits interest on the current balance.

**29** With reference to your code in the previous question, explain *encapsulation*, and why the balance should be private. **3 marks**

**30** Describe how a development team could use a Kanban board and daily stand-up meetings to manage a software project. **4 marks**

**31** A developer is building an app that tracks students' locations so parents can see when their child arrives at school. Discuss the social, ethical and legal issues the developer must consider. **5 marks**

**32** Design a secure login system for a web app. Describe the process from a user registering to their authenticated session, justifying the security measure used at each step. **6 marks**

# End of paper

# Solutions and marking guidelines

Written for Trialbank. Not official NESA marking guidelines.

## Section I

| Q | Answer | Why |
|---|---|---|
| 1 | B | Non-functional requirements describe how well the system works (performance, security, usability), not what it does. |
| 2 | A | Without a salt, identical passwords have identical hashes, so precomputed tables of hashes can reverse them. |
| 3 | B | HSTS (Strict-Transport-Security) makes the browser refuse plain HTTP connections to that site. |
| 4 | B | f is the Fibonacci function: 0, 1, 1, 2, 3, 5, 8, so f(6) = 8. |
| 5 | B | Testing on data the model never trained on shows whether it generalises or has just memorised the training set. |
| 6 | B | The space is the descendant combinator: any .card inside a section. section.card would mean a section that itself has the class. |
| 7 | A | The different messages let an attacker find out which usernames are real. A single generic message prevents this. |
| 8 | B | MLOps applies DevOps ideas (automation, continuous delivery, monitoring) to machine learning models. |
| 9 | A, B, D, F | REST uses stateless HTTP requests to URL-named resources, usually returning JSON. It doesn't need WebSockets and isn't limited to HTML. |
| 10 | A, B, D, F | self.wheels finds Motorbike's class attribute (2) first. Motorbike inherits both __init__ and describe(); it overrides only the wheels attribute. |
| 11 | A, B, D | Bias, consent and surveillance are ethical issues. Speed and power use are not. |
| 12 | B, C, F | bcrypt, Argon2 and PBKDF2 are slow, salted password hashes. Base64 is reversible encoding, and MD5 is fast and broken. |
| 13 | 1–C, 2–D, 3–A, 4–B (2 marks: all correct; 1 mark: at least half correct) | |
| 14 | 1–C, 2–A, 3–B, 4–E, 5–D (2 marks: all correct; 1 mark: at least half correct) | |
| 15 | 1–B, 2–D, 3–E, 4–A, 5–C (2 marks: all correct; 1 mark: at least half correct) | |
| 16 | 1–B, 2–C, 3–A, 4–D, 5–F, 6–E (2 marks: all correct; 1 mark: at least half correct) | |

## Section II

**17 (2 marks)**

| Criteria | Marks |
|---|---|
| Outlines that a unique random salt makes identical passwords hash differently, defeating precomputed (rainbow) tables | 2 |
| Identifies a feature of salting | 1 |

**Sample answer**

A salt is a random value generated for each user and combined with the password before hashing, and stored with the hash. Two users with the same password end up with different hashes, so an attacker can't use precomputed rainbow tables or crack many accounts at once; each password must be attacked separately.

### 18 (3 marks)

| Criteria | Marks |
|---|---|
| Explains how the attack uses the victim's logged-in session, and describes a valid defence | 3 |
| Describes the attack or a defence | 2 |
| Identifies a feature of CSRF | 1 |

**Sample answer**

The victim is logged in to a site, so their browser holds its session cookie. While logged in, they visit an attacker's page that secretly submits a request to that site (for example, a hidden form that changes their email or transfers money). The browser automatically attaches the session cookie, so the site treats the forged request as the user's own. **Prevention:** include a secret, unpredictable CSRF token in every form and reject requests without it; the attacker's page can't read the token. Setting cookies to SameSite also stops them being sent with cross-site requests.

### 19 (4 marks)

| Criteria | Marks |
|---|---|
| (a) Identifies reflected XSS and explains that script in the q parameter runs in the victim's browser | 2 |
| (a) Identifies XSS | 1 |
| (b) Escapes the output (escape() or an auto-escaping template) | 2 |
| (b) Partially correct fix | 1 |

**Sample answer**

(a) Reflected cross-site scripting. The search text is put straight into the HTML. An attacker sends a victim a link such as `/search?q=<script>…</script>`; the script runs in the victim's browser on this site, where it can steal their session cookie or act as them.

```
from markupsafe import escape

@app.route("/search")
def search():
    q = request.args.get("q", "")
    return f"<h1>Results for {escape(q)}</h1>"
```

Also accept render_template with a Jinja template, which escapes automatically.

### 20 (4 marks)

| Criteria | Marks |
|---|---|
| Describes the request, server-side processing, database query and response in order, with the role of each tier | 4 |
| Describes most steps | 3 |
| Outlines some steps | 2 |
| Provides some relevant information | 1 |

**Sample answer**

The **browser** sends an HTTPS GET request for /profile, with the session cookie that identifies the student. The **web server** receives it, checks the session to authenticate the student, and runs the route's code. That code sends a parameterised SQL query to the **database server**, which returns the student's row (name, year, results). The web server inserts the data into an HTML template and sends the page back in an HTTP response with status 200. The browser then renders the HTML, requests the CSS, JavaScript and images it references, and displays the page.

### 21 (4 marks)

| Criteria | Marks |
|---|---|
| Compares both, giving clear similarities and differences with examples (installation, offline use, device access, performance, cost) | 4 |
| Describes features of both | 3 |
| Outlines features of one or both | 2 |
| Provides some relevant information | 1 |

**Sample answer**

**Similarities:** both can be installed on the home screen, open full screen, send notifications and work offline (a PWA through a service worker cache). **Differences:** a PWA is one web codebase that runs on any device through the browser and updates instantly without an app store, so it's cheaper to build and maintain. A native app is written for each platform (Swift for iOS, Kotlin for Android), is distributed through app stores, and gets fuller access to device hardware (Bluetooth, sensors, background tasks)

and usually better performance. A PWA suits content and tools that need wide reach at low cost; a native app suits games or apps that rely heavily on device features.

### 22 (5 marks)

| Criteria | Marks |
|---|---|
| Correct function that passes all tests, including ties and short lists | 5 |
| Correct function with one minor error | 4 |
| Sorts correctly but mishandles ties or n | 3 |
| Partially working logic | 2 |
| Some relevant code | 1 |

**Sample answer**

```
def top_students(results, n):
    ranked = sorted(results, key=lambda r: (-r[1], r[0]))
    return [name for name, mark in ranked[:n]]
```

Any correct method earns full marks, for example sorting by name and then by mark with reverse=True. Tests: the example above; n larger than the list; an empty list gives [].

### 23 (3 marks)

| Criteria | Marks |
|---|---|
| Correct join, WHERE, GROUP BY, HAVING and ORDER BY | 3 |
| Query with one error or omission | 2 |
| Some correct SQL | 1 |

**Sample answer**

```
SELECT Bookings.room, COUNT(*) AS bookings
FROM Bookings
JOIN Students ON Bookings.student_id = Students.id
WHERE Students.year = 11
GROUP BY Bookings.room
HAVING COUNT(*) > 5
ORDER BY bookings DESC;
```

WHERE filters rows before grouping; HAVING filters the groups after counting.

### 24 (2 marks)

| Criteria | Marks |
|---|---|
| Explains that branches isolate work so it can be developed and tested without affecting the main code, then merged | 2 |

| | |
|---|---|
| Identifies a feature of branches | 1 |

**Sample answer**

A branch is a separate line of development. Each feature or bug fix is built and tested on its own branch, so the main branch always stays stable and working. Several developers can work in parallel without overwriting each other, and when a feature is finished and reviewed it is merged back into main. If an idea fails, the branch can simply be abandoned.

**25 (4 marks)**

| Criteria | Marks |
|---|---|
| (a) Explains that the student's features are tested at each node from the root, following branches until a leaf gives the prediction | 2 |
| (a) Outlines a feature of decision trees | 1 |
| (b) Valid advantage and valid disadvantage | 2 |
| (b) One of these | 1 |

**Sample answer**

(a) The tree is a series of yes/no questions learnt from training data. Starting at the root, the new student's data is tested (for example, "attendance ≥ 85%?"), and the answer decides which branch to follow to the next question ("homework completion ≥ 70%?"). This continues until a leaf node is reached, which gives the prediction: pass or fail.

(b) **Advantage:** it's easy to explain, since the exact rules behind each prediction can be shown to teachers and students, and it works well with small datasets. **Disadvantage:** it can overfit by growing too many specific branches, and it's usually less accurate than a neural network on complex patterns.

**26 (4 marks)**

| Criteria | Marks |
|---|---|
| Explains how bias can enter at several stages (data collection, labelling, design or training, deployment) with examples | 4 |
| Describes bias at two stages | 3 |
| Outlines how bias can occur | 2 |
| Provides some relevant information | 1 |

**Sample answer**

**Data collection:** if the training data under-represents some groups (for example, mostly lighter-skinned faces), the model performs worse for others. **Historical data:** past decisions that were unfair, such as hiring records, teach the model to repeat them. **Labelling:** human labellers bring their own assumptions. **Design and training:** choosing features that act as proxies for protected attributes

(postcode for ethnicity), or optimising only overall accuracy, hides poor results for minorities. **Deployment:** using a model on a population different from its training data, or feedback loops where its decisions shape future data, can amplify bias over time.

### 27 (2 marks)

| Criteria | Marks |
|---|---|
| Correct accuracy (85%) and precision (0.8 or 80%) with working | 2 |
| One correct | 1 |

**Sample answer**

Accuracy = correct predictions ÷ total = (40 + 45) ÷ 100 = **85%**.

Precision = true positives ÷ everything flagged as spam = 40 ÷ (40 + 10) = **0.8 (80%)**.

### 28 (5 marks)

| Criteria | Marks |
|---|---|
| Both classes correct: private balance, validation with ValueError, subclass extends BankAccount, calls super().__init__ and reuses deposit | 5 |
| Mostly correct with one error | 4 |
| BankAccount correct, SavingsAccount partly correct | 3 |
| BankAccount partly correct | 2 |
| Some relevant code | 1 |

**Sample answer**

```
class BankAccount:
    def __init__(self):
        self.__balance = 0

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Deposit must be positive")
        self.__balance += amount

    def withdraw(self, amount):
        if amount <= 0 or amount > self.__balance:
            raise ValueError("Invalid withdrawal")
        self.__balance -= amount

    def get_balance(self):
        return self.__balance


class SavingsAccount(BankAccount):
    def __init__(self, interest_rate):
```

```
        super().__init__()
        self.interest_rate = interest_rate

    def add_interest(self):
        self.deposit(self.get_balance() * self.interest_rate)
```

**29 (3 marks)**

| Criteria | Marks |
|---|---|
| Explains encapsulation, links the private balance to controlled access through methods, and justifies it with reference to the code | 3 |
| Explains encapsulation with some link to the code | 2 |
| Outlines encapsulation | 1 |

**Sample answer**

Encapsulation bundles an object's data with the methods that act on it, inside the class. The balance is private (`self.__balance`), so code outside the class can't change it directly; the only way to change it is by sending a message to the object through `deposit()` or `withdraw()`, which validate every change. This means the balance can never become negative or be set to an invalid value by mistake, and the subclass SavingsAccount also has to go through `deposit()` to add interest. `get_balance()` gives read-only access.

**30 (4 marks)**

| Criteria | Marks |
|---|---|
| Describes both tools in detail and how they help the team track progress and remove blockers | 4 |
| Describes both tools | 3 |
| Describes one tool | 2 |
| Provides some relevant information | 1 |

**Sample answer**

A **Kanban board** shows every task as a card in columns such as To do, In progress, Testing and Done. Team members pull the next card when they're free and move it across as work progresses, so everyone can see what's happening and where work is piling up. Limiting how many cards can be in progress at once stops people starting too much and finishing too little. A **daily stand-up** is a short meeting (about 15 minutes) where each member says what they finished yesterday, what they'll do today, and anything blocking them. Blockers are raised early so they can be solved quickly, and the board is updated during the meeting so it stays accurate.

**31 (5 marks)**

| Criteria | Marks |
|---|---|
| Discusses social, ethical and legal issues in detail, with arguments for and against and specific measures | 5 |
| Describes issues in all three areas | 4 |
| Describes issues in two areas | 3 |
| Outlines some issues | 2 |
| Provides some relevant information | 1 |

**Sample answer**

**Legal:** location data is personal information under the Privacy Act 1988 and the Australian Privacy Principles, so the app must collect only what it needs, explain how it's used, store it securely and get valid consent, which for minors involves parents. A data breach could also trigger mandatory notification obligations. **Ethical:** constant tracking can harm students' autonomy and trust, and they should know when they're tracked and have a say in it. The data could be misused, for example by an abusive family member, so location history should be limited (arrival notifications rather than continuous tracking) and deleted quickly. **Social:** parents gain peace of mind and schools may improve safety, but normalising surveillance could change how young people feel about privacy, and students without smartphones could be left out. On balance, the app is reasonable if it follows privacy by design: minimal data, clear consent, encryption and strict retention limits.

**32 (6 marks)**

| Criteria | Marks |
|---|---|
| Describes registration, login and session steps in detail, each with a justified security measure | 5–6 |
| Describes most steps with some justification | 3–4 |
| Outlines some steps or measures | 1–2 |

**Sample answer**

**Registration:** the form is served over HTTPS, so details can't be read in transit. The server validates the email and enforces a strong password policy (length, checked against known breached passwords). The password is hashed with a slow, salted algorithm such as bcrypt or Argon2 and only the hash is stored, so a stolen database doesn't reveal passwords. A verification email confirms the address. **Login:** the server looks the user up with a parameterised query (preventing SQL injection) and compares the hash. Error messages are generic ("Incorrect email or password") to prevent user enumeration, and failed attempts are rate-limited or locked out to stop brute-force attacks. Multi-factor authentication (an authenticator app code) protects accounts even if the password is stolen. **Session:** after login the server creates a random session ID and sends it in a cookie with Secure, HttpOnly and SameSite set, protecting it from interception, XSS theft and CSRF. The ID is regenerated at login, sessions expire after inactivity, and logout destroys the session on the server. Logins are logged so suspicious activity can be detected.